

CM Track 21: Auth, Sessions und Mandantentrennung

Datum: 2026-08-28 **Stack:** Next.js 15, React 19, TypeScript 5.9, Supabase SDK 2.98, Stripe 20.4
Ausgangsstand: 0d00473 (Track 20 Nachtrag) **Tests:** 1457 → 1508 (51 neue), alle grün **Typecheck:** tsc --noEmit ohne Fehler · **Lint:** 0 Fehler, 20 Warnungen (unveränderter Bestand) · **Build:** next build kompiliert fehlerfrei (✓ Compiled successfully); die anschließende statische Erzeugung läuft lokal ohne SUPABASE_SERVICE_ROLE_KEY nicht durch — der bekannte lokale Zustand, unverändert gegenüber HEAD

Zusammenfassung

Geprüft wurde, **wem die Anwendung glaubt und wie lange:** das Sitzungs-Cookie, der zweite Faktor, die Rollenvergabe, die Trennung zwischen zwei Salons, die Registrierungs- und Reset-Wege, die Autorisierung jeder API-Route und die Stripe-Webhook-Signatur.

Sieben Befunde, alle behoben. Sie liegen alle auf derselben Linie und diese Linie ist neu für die Tracks 11–20: die bisherigen Befunde betrafen fast immer **Daten** — was jemand sehen, ändern oder auslösen konnte. Die schwersten Befunde hier betreffen **Zeit:** wie lange ein einmal erteiltes Recht gilt, nachdem der Grund dafür weggefallen ist.

Die beiden schwersten:

1. **Zwei-Faktor-Authentifizierung ließ sich mit einem einzigen POST abschalten** — ohne aktuellen Code, ohne Passwort, nur mit dem Sitzungs-Cookie. 2. **Ein Passwortwechsel beendete keine einzige offene Sitzung.** Das Cookie läuft 365 Tage und hing am Passwort nicht. Wer sein Passwort ändert, *weil* jemand anderes in seinem Konto ist — der häufigste Grund überhaupt —, hat diesen Jemand damit nicht ausgesperrt. Beim Passwort-Reset kam eine zweite Ursache dazu: er läuft vollständig im Browser gegen Supabase-Auth, der Server erfährt davon gar nichts.

Beide zusammen ergaben eine Kette, in der jede Reaktion auf eine Kompromittierung ins Leere lief: 2FA war abschaltbar, das Passwort war änderbar, aber keine der beiden Maßnahmen entfernte den Angreifer.

Die Mandantentrennung selbst — Salon A gegen Salon B — hat sich als weitgehend solide erwiesen. Die Tracks 13–18 haben dort gründlich vorgearbeitet: alle Anbieter-Routen lösen den Salon aus der Session auf (`getOwnedSalon`) statt aus dem Request, und die Schreibpfade tragen den Besitznachweis als `.eq('salon_id', ...)` in der Query selbst. **Zwei Lücken blieben, beide auf derselben Spalte:** `staff_id` wurde nie gegen den Salon gehalten.

Befundübersicht

ID	Befund	Schwere	Status
CM21-01	2FA mit einem POST abschaltbar	HOCH (P1)	behoben
CM21-02	Passwortwechsel/-Reset beenden keine Sitzungen	HOCH (P1)	behoben
CM21-03	Sitzungs-Cookie ohne <code>__Secure-</code> -Präfix	MITTEL (P2)	behoben
CM21-04	<code>admin</code> konnte <code>super_admin</code> herabstufen	MITTEL (P2)	behoben
CM21-05	<code>bookings.staff_id</code> ungeprüft (Mandantentrennung)	MITTEL (P2)	behoben
CM21-06	<code>/api/recommendations</code> : tote Spalte + ungeprüfte IDs	MITTEL (P2)	behoben
CM21-07	Super-Admin-Upload wählte freien Pfad im Bucket	NIEDRIG (P3)	behoben

Dazu ein Wahrhaftigkeits-Befund ohne eigene ID (siehe „Nachtrag: die Kachel, die log“).

CM21-01: Zwei-Faktor-Authentifizierung ließ sich mit einem POST abschalten (HOCH)

Datei: `src/app/api/auth/2fa/setup/route.ts`

Beweis

`POST /api/auth/2fa/setup` schrieb bedingungslos:

```
await supabase.from('user_2fa').upsert(  
  { user_id: session.user.id, secret, enabled: false, updated_at: ... },  
  { onConflict: 'user_id' },  
)
```

Auf einer Zeile mit `enabled = true` ist das **keine Einrichtung, sondern eine Abschaltung**. Verlangt wurde dafür: nichts. Kein aktueller TOTP-Code, kein Passwort, keine erneute Anmeldung — allein das Sitzungs-Cookie.

Der Login prüft den zweiten Faktor seit Track 17 über genau diese Spalte (`authorizeCredentials` in `auth.config.ts`, `twoFa?.enabled === true`). Nach einem einzigen POST steht dort `false`, und die Anmeldung fragt wieder nur nach dem Passwort — bei einem Konto mit 2FA gerade der Teil, den man als kompromittiert annehmen muss.

Eine Route zum **absichtlichen** Deaktivieren gibt es in der gesamten Anwendung nicht. Das hier war eine versehentliche.

Zweiter, leiserer Schaden: wer aus Neugier ein zweites Mal auf „Aktivieren“ tippte und den neuen Code nie bestätigte, stand ohne 2FA da — ohne jeden Hinweis darauf.

Fix

Eine aktive 2FA wird von dieser Route nicht mehr angefasst: Vorab-Lesen des Zustands, bei `enabled === true` Antwort **409** und **kein Schreibvorgang**. Ein Lesefehler sperrt ebenfalls (fail closed) — bei

unbekanntem Zustand darf der Upsert nicht laufen.

Bewusste Abgrenzung: ein Wechsel des Geheimnisses (neues Telefon) braucht einen eigenen Endpunkt, der den *aktuellen* Code prüft — und dafür eine Spalte für das noch unbestätigte Geheimnis, die `user_2fa` live nicht hat. Eine Rotation, die bei Abbruch ohne zweiten Faktor endet, wäre der schlechtere Zwischenstand als ein Riegel, der nichts kaputt macht. Der Wechsel läuft vorerst über den Support.

CM21-02: Ein Passwortwechsel beendete keine offene Sitzung (HOCH)

Dateien: `src/modules/auth/session.ts`, `src/modules/auth/auth.config.ts`,
`src/app/api/auth/change-password/route.ts`, `src/app/api/auth/session-revoke/route.ts` (neu),
`src/app/(auth)/auth/reset-password/page.tsx`

Beweis

Das Sitzungs-Cookie läuft **365 Tage** (`session.maxAge` in `auth.config.ts`) und wird alle 24 Stunden rollend erneuert. `getServerSession` prüft seit Track 17 bei jedem Aufruf Rolle und Sperre gegen `profiles` — die Aussage „*dieses Passwort gilt nicht mehr*“ stand aber **nirgends**, weder im Token noch in der Datenbank.

Zwei Wege, beide wirkungslos:

Weg	Was passierte	Was NICHT passierte
POST <code>/api/auth/change-password</code>	Passwort in <code>auth.users</code> geändert, <code>password_must_change</code> gelöscht, die eigene Sitzung abgemeldet	jede andere offene Sitzung des Kontos lief weiter — bis zu 365 Tage
<code>/auth/reset-password</code> („Passwort vergessen“)	<code>supabase.auth.updateUser({ password })</code> im Browser , mit dem Anon-Key	der Server erfuhr vom Wechsel überhaupt nichts; das NextAuth-Cookie blieb unberührt

Der zweite Weg ist der schwerere, weil er der ist, den ein Betroffener wählt, wenn er nicht mehr in sein Konto kommt. ChairMatch hat zwei Anmeldesysteme nebeneinander (NextAuth stellt das Cookie aus, Supabase-Auth hält das Passwort); der Reset spricht nur das zweite an.

Fix

Ein **Sitzungs-Widerruf mit Zeitstempel**:

1. `auth.config.ts` legt beim **Login** `token.loginAt` an. Bewusst nicht `token.iat`: der Rolling-Refresh stellt den Token alle 24 Stunden neu aus und setzt `iat` dabei neu — eine gestohlene, weiterbenutzte Sitzung hätte sich damit selbst an jedem Widerruf vorbeigeschoben. Der `if (user)` -Zweig läuft nur beim Login, der Refresh reicht den Wert unverändert durch. 2. `getServerSession` liest den jüngsten Widerruf des Kontos und verwirft jede Sitzung, deren `loginAt` davor liegt. Ein Token ohne `loginAt` (aus der Zeit vor diesem Track) gilt als älter als jeder Widerruf. Fail closed: ist der Widerruf nicht lesbar, gibt es keine Sitzung — dieselbe Linie wie beim Kontostand. 3. `/api/auth/change-password` schreibt den Widerruf und meldet **ehrlich** zurück, ob er gespeichert werden konnte (`sessionsRevoked`); schlägt er fehl, kommt eine ausdrückliche Warnung statt eines

stillen `success: true`. 4. **Neu:** `POST /api/auth/session-revoke` — das serverseitige Ende des Supabase-Resets. Die Reset-Seite ruft es nach erfolgreichem Passwort-Update auf. Der Ausweis ist das Supabase-Zugangstoken der Reset-Sitzung; es wird **nicht geglaubt, sondern geprüft** (`admin.auth.getUser(jwt)`), und die Nutzer-ID kommt aus der Antwort, nie aus dem Request. Rate-Limit 5 / 15 min / IP.

Warum der Widerruf in `audit_logs` steht und nicht in `profiles`: der richtige Ort wäre eine Spalte `profiles.sessions_valid_from`. Die gibt es live nicht, und ChairMatch hat weder einen Migrations-Runner noch einen DB-Zugang für den Deploy. Eine Auswahl auf eine fehlende Spalte beantwortet PostgREST mit `42703` — **für die ganze Abfrage**. Der Kontostand wäre damit unlesbar und jede Sitzung sofort weg. `audit_logs` ist live vorhanden, wird von `/admin/audit-logs` ohnehin gelesen und trägt mit `user_id`, `action` und `created_at` genau die drei Felder, die gebraucht werden. Die Aktion heißt `SESSION_REVOKED` und ist damit auch für den Admin sichtbar.

Operative Folge: ein Passwortwechsel meldet ab sofort **alle** Geräte ab, auch das eigene. Das ist Absicht — ein Wechsel, der die aufrufende Sitzung ausnimmt, wäre für den Angreiferfall die falsche Ausnahme.

CM21-03: Das Sitzungs-Cookie trug keinen `__Secure-`-Präfix (MITTEL)

Datei: `src/modules/auth/auth.config.ts`

Beweis

```
cookies: { sessionToken: { name: 'authjs.session-token', ... } }
```

Das war nicht nur eine Umbenennung des Auth.js-Standards, es hat dessen einzigen Zusatzschutz entfernt. Auth.js nennt das Cookie in einer HTTPS-Umgebung von sich aus `__Secure-authjs.session-token`, und dieses Präfix ist eine Regel, die **der Browser durchsetzt**: ein so benanntes Cookie nimmt er nur über HTTPS und nur mit `Secure` entgegen.

Ohne Präfix ist der Name gewöhnlich — und **Cookie-Setzen kennt keine Herkunftstrennung**: eine beliebige Subdomain von `chairmatch.de`, auch eine über reines HTTP ausgelieferte, auch eine, die jemand anders betreibt, kann `authjs.session-token` mit `Domain=.chairmatch.de` setzen und das echte Cookie überschreiben. Das ist der klassische Session-Fixation-Weg: der Angreifer setzt *seinen* Token, das Opfer arbeitet in dessen Sitzung weiter und legt dort Daten an.

Das `secure`-Flag schützt davor **nicht** — es regelt, wohin der Browser das Cookie sendet, nicht, wer es setzen darf.

Fix

Der Name hängt jetzt an derselben Bedingung wie `secure: __Secure-authjs.session-token` in Produktion, ohne Präfix in der Entwicklung (über `http://localhost` würde der Browser das Präfix-Cookie sonst ablehnen und kein Login mehr funktionieren).

Operative Folge: der Cookie-Name ändert sich. **Alle bestehenden Sitzungen sind mit dem Deploy einmalig ungültig, jeder muss sich neu anmelden.** Das ist zugleich der saubere Übergang für CM21-02: nach dem Deploy trägt jeder Token ein `loginAt`.

CM21-04: Ein `admin` konnte jeden `super_admin` herabstufen (MITTEL)

Datei: `src/app/api/admin/route.ts`, Aktion `user-role`

Beweis

Der Riegel sah nur nach oben:

```
if (['admin', 'super_admin'].includes(role) && callerRole !== 'super_admin') {  
  return 403 // „Nur super_admin darf Admin-Rollen vergeben“  
}
```

Geprüft wurde die **neue** Rolle, nie die **bestehende** des Ziels. Nach unten war die Route offen: ein `admin` konnte jeden `super_admin` der Plattform auf `kunde` setzen. Die Rollen-Nachprüfung aus Track 17 macht das binnen 15 Sekunden in jeder laufenden Sitzung wirksam.

Selbst hochstufen konnte er sich damit nicht — aber er konnte **die einzige Rolle abräumen, die ihn hätte zurückstufen können**, und die Super-Admin-Bereiche (`/admin/super/*`: Einstellungen, Kategorien, Logo, Onboarding) unbesetzt lassen.

Zweiter Punkt derselben Stelle: niemand war gegen die eigene Aussperrung geschützt. Ein `super_admin` konnte sich selbst auf `kunde` setzen — die Änderung, die danach niemand mehr rückgängig machen kann.

Fix

Die Rolle des **Ziels** wird gelesen und geprüft: ist sie `admin` oder `super_admin`, darf nur ein `super_admin` sie entziehen. Fail closed — ist das Ziel nicht lesbar, wird nichts geändert; ein unbekanntes Ziel ist jetzt 404 statt einer stillen Nulländerung. Die eigene Rolle lässt sich über diese Route nicht mehr ändern.

CM21-05: `bookings.staff_id` wurde nie gegen den Salon gehalten (MITTEL)

Datei: `src/modules/booking/booking.actions.ts`

Beweis

`createBooking` schrieb:

```
staff_id: data.staffId || null,
```

`staffId` kam aus dem Request und wurde nie geprüft. Track 18 hat dieselbe Lücke für `serviceId` geschlossen („die günstige Leistung eines fremden Salons zum Preis von dort“) — `staffId` blieb daneben stehen.

`staff` ist eine mandantengetrennte Tabelle (`staff.salon_id`, bestätigt durch die Spaltensonde in `src/test/live-schema.ts`). Eine fremde ID zu setzen heißt, den Termin eines Betriebs auf eine Person eines **anderen** Betriebs zu schreiben.

Was daran hängt: `getAvailableSlots(salonId, date, duration, staffId)` in `src/lib/scheduling.ts` filtert die Belegung bereits auf `staff_id` — eine Terminplanung pro Person ist vorgesehen. Ab dem Tag, an dem sie eingeschaltet wird, wäre eine fremde ID nicht nur eine falsche Zeile, sondern ein Hebel in den Kalender eines fremden Salons: Termine bei Salon A würden die Verfügbarkeit einer Person bei Salon B verbrauchen.

Ein Fremdschlüssel allein fängt das nicht ab: er prüft, **dass** es die Person gibt, nicht, **wessen** Person sie ist.

Fix

`staffId` wird gegen `staff.salon_id === data.salonId` und `is_active` geprüft, bevor die Zeile entsteht. Fremd und unbekannt bekommen **dieselbe** Antwort — welche Mitarbeitenden ein fremder Betrieb hat, geht den Aufrufer nichts an. Fail closed bei Lesefehler (503).

CM21-06: `/api/recommendations` lief in eine tote Spalte (MITTEL)

Datei: `src/app/api/recommendations/route.ts`

Beweis

```
.select('id, user_id, salon_id, salons!inner(owner_id)')
```

bookings.user_id existiert live nicht — die Spalte heißt `customer_id` (Spaltensonde, `src/test/live-schema.ts`). PostgreSQL beantwortet eine unbekannte Spalte mit `42703`, der Fehler landete in `bookingErr`, und **jeder** Aufruf — auch der vollkommen richtige des Saloninhabers — bekam „Buchung nicht gefunden“ (404).

Das Anlegen einer Produktempfehlung war damit seit jeher unmöglich, und die Fehlermeldung zeigte in die falsche Richtung. Die Folge für die Sicherheit ist die unangenehmere: **die**

Autorisierungsprüfung darunter ist nie gelaufen. Was Track 17 an dieser Route repariert hat (`customerId` aus der Buchung statt aus dem Request), war ungetestete Theorie — der Pfad hat den Prüfblock nie erreicht.

Auf demselben Weg zwei ungeprüfte Felder:

- **staffId** — dieselbe Lücke wie CM21-05. Gelesen wird die Spalte in `getRecommendationsForCustomer` als Einbettung `staff(name, title)`: ein Anbieter konnte Name und Funktion einer Person aus einem **fremden** Salon in eine Empfehlung schreiben, die seiner eigenen Kundin angezeigt wird.
- **productId** — eine erfundene ID lief in `23503` und kam als 500 zurück, eine ausgelistete führte zu einer Empfehlung, die ins Leere zeigt.

Fix

`customer_id` statt `user_id`; ein Lesefehler ist jetzt 500 und nicht mehr als 404 getarnt. `staffId` wird gegen den Salon der Buchung geprüft, `productId` gegen Existenz und `is_active`.

Bewusst keine Eingrenzung von `productId` auf den eigenen Salon: der Shop ist eine gemeinsame Fläche, und ob ein Anbieter nur eigene Ware empfehlen darf, ist eine Produktentscheidung — keine,

die ein Härte-Track still trifft.

CM21-07: Der Super-Admin-Upload wählte einen freien Pfad im Bucket (NIEDRIG)

Datei: `src/modules/super-admin/super-admin.actions.ts`

Beweis

```
const folder = (formData.get('folder') as string) || 'uploads'
const path = `${folder}/${Date.now()}.${ext}`
```

Der Bucket steht seit Track 18 auf einer Positivliste — der Pfad **darin** war frei wählbar. `folder = '../salon-images/logos'` oder ein Name mit Schrägstrichen legte die Datei irgendwo im Bucket ab, auch dort, wo fachliche Dateien liegen.

Niedrig eingestuft, weil die Aktion `super_admin` verlangt: die höchste Rolle der Anwendung, und diese Rolle kann ohnehin mehr. Der Fix ist trotzdem richtig, weil ein Pfad aus einem Formularfeld nie ein Pfad sein sollte.

Fix

Ein einzelnes, flaches Segment aus Buchstaben, Ziffern, Strich und Unterstrich — mehr hat nie jemand gebraucht.

Nachtrag: die Kachel, die log

Datei: `src/app/(protected)/account/page.tsx`

Beim Prüfen von CM21-01 fiel die Oberfläche dazu auf. Die 2FA-Kachel setzte nach dem Klick auf „Aktivieren“ ihren Zustand auf `enabled = true` und zeigte „Aktiv“ — geschrieben hatte die Route zu diesem Zeitpunkt aber `enabled: false`. Wahr wird das erst mit einem gültigen Code aus `/api/auth/2fa/verify`, und **einen Ort, an dem dieser Code hätte eingegeben werden können, gab es in der gesamten Oberfläche nicht.**

Der Nutzer las also „Aktiv“, hielt sein Konto für zweifach gesichert und meldete sich danach weiter allein mit seinem Passwort an. Eine behauptete Schutzwirkung ist schlimmer als eine fehlende: sie verhindert, dass jemand das echte Loch bemerkt.

Die Kachel hat jetzt den zweiten Schritt: Geheimnis anzeigen, Code eingeben, bestätigen. „Aktiv“ steht erst da, wenn der Server es sagt.

Ohne Befund

Geprüft und in Ordnung befunden:

Session-Handling

- `getSession` liest Rolle, Sperre und Löschestand bei jedem Aufruf aus `profiles`, mit 15-Sekunden-Cache und Obergrenze; fail closed bei Lesefehler.
- Cookie-Flags `httpOnly`, `sameSite=lax`, `secure` in Produktion — korrekt gesetzt (der Name war der Befund, nicht die Flags).
- Demo-Konten sind durch ein doppeltes Gate (`NODE_ENV` **und** kein `VERCEL`) aus jeder Deploy-Umgebung ausgeschlossen.
- Login-Rate-Limit 10 Fehlversuche / 15 min pro IP-HMAC, das `throw` liegt korrekt außerhalb des eigenen `try`.
- 2FA-Prüfung im Login-Pfad (`authorizeCredentials`) greift; kein Konto-Orakel in `/api/auth/2fa/status`.
- Keine Rolle aus `user_metadata` — der Weg zur frei gewählten Rolle ist seit Track 13 zu.

Mandantentrennung (Salon-Isolation)

- Alle Anbieter-Routen (`/api/provider/*`, `/api/me/*`) lösen den Salon über `getOwnedSalon(supabase, session.user.id)` auf; der Salon steht nie im Request.
- `services`, `products`, `rental_equipment`: Schreibpfade tragen den Besitznachweis als `.eq('salon_id', ...)` **in der Query** und melden 404 statt eines stillen Erfolgs, wenn nichts getroffen wird.
- `compliance`, `owner/documents`, `owner/authorities-pack`: jede Route lädt `salons.owner_id` und hält ihn gegen die Session, bevor sie liest oder schreibt.
- `messages` / `conversations`: Mitgliedschaft in `conversation_participants` wird geprüft, 403 statt 404 (die Existenz eines Fadens ist selbst eine Auskunft).
- `bookings`, `rental_bookings`, `rental_requests`: Berechtigung wird aus der echten Beziehung abgeleitet (Kunde / Inhaber / Admin), nicht aus „ist nicht fremd“.
- `notifications`, `favorites`, `cart`, `orders`, `payout-account`, `tenant-profile`: Besitzer kommt ausschließlich aus der Session.
- `uploads/[id]`: nicht-öffentliche Dateien (Zertifikate) verlangen Eigentümer oder Admin; DELETE verlangt den Eigentümer.

IDOR auf Salon-Ebene

- `/api/salons/[id]` liefert eine Positivliste ohne `owner_id`, `email` und Moderationszustand; `staff` ohne `user_id`.
- `/api/stripe/checkout` grenzt jeden Zweig auf die eigene Buchung / Bestellung / Miete ein.
- UUID-Prüfung auf allen `[id]`-Routen (Tracks 18/19), damit eine Falscheingabe nicht als 500 oder irreführendes 404 endet.

Rollen-Eskalation

- Rollen-Leiter in `src/lib/rbac.ts`: `INVESTOR` steht bewusst neben der Leiter, nicht darin.
- Middleware-RBAC deckt Seiten- **und** API-Präfixe ab; die Route prüft zusätzlich selbst.
- `/api/setup/promote-admin`: Rate-Limit, zeitkonstanter Vergleich, Mindestlänge des Schlüssels.
- `/admin/super/*` ist serverseitig über `requireRole(['super_admin'])` im Layout abgesichert, nicht nur über die Middleware.

Einladungs-/Registrierungsflow

- **Es gibt keinen Einladungsflow** — weder Tabelle, Route noch Seite. Der geprüfte Punkt entfällt damit; es gibt keine Invite-Token, die sich wiederholen oder ablaufen könnten.
- `/api/auth/register` und `/api/register-provider` : Rate-Limit pro IP **und** pro Adresse, Rolle wird serverseitig gesetzt, das Aufräumen bei fehlgeschlagenem Salon-Insert löscht das Auth-Konto zuerst.
- `/api/auth/forgot-password` : identische Antwort in allen Fällen, kein Konto-Orakel.

API-Middleware

- Alle 106 Route-Dateien durchgesehen (Autorisierungs-Landkarte je Datei, die sicherheitsrelevanten vollständig gelesen). Ohne Session-Prüfung sind nur die Routen, die ausdrücklich öffentlich sein sollen (Analytics-Sammler, Newsletter-Anmeldung, öffentliche Produkt- und Salondaten, Verfügbarkeit, Cookie-Consent, Wait-List, Match-Finder, CSP-Reports, IndexNow) sowie die Cron-Routen (eigener Schlüssel) und der Stripe-Webhook (Signatur).
- Default-Deny der Middleware für alles außerhalb der Positivliste ist intakt.

RLS-Policies

- Auf Code-Ebene geprüft. Anzumerken ist der **Grundzustand**, nicht ein neuer Befund: der Produktivcode arbeitet durchgängig mit dem Service-Role-Client und umgeht RLS damit bewusst. Die Mandantentrennung liegt deshalb **im Code**, nicht in der Datenbank — genau deshalb ist sie hier Zeile für Zeile geprüft worden.
- Eine Aussage über den *live angewendeten* Policy-Bestand ist von hier aus nicht möglich: Agents haben keinen DB-Zugang (kein Supabase-MCP, `psql` blockiert), und `supabase/migrations/*` bildet den Live-Stand nachweislich nicht ab. Aufgefallen ist dabei, dass die Policies in `20260514_security_hardening_qls.sql` für `documents` und `authorities_packs` auf `owner_id` filtern, während der Produktivcode dort live `salon_id` schreibt — **ungeprüft, welcher der beiden Stände in der Produktionsdatenbank steht**. Das gehört auf die Liste für einen Track mit DB-Zugang und wird hier nicht als behoben ausgegeben.

Stripe-Webhook

- `POST /api/stripe/webhook` liest den Rohtext (`req.text()`), verlangt den `stripe-signature` -Header, verweigert ohne konfiguriertes `STRIPE_WEBHOOK_SECRET` den Dienst und prüft mit `stripe.webhooks.constructEvent` . Ein gefälschtes Ereignis fällt durch. Die Zeitstempel-Toleranz der Signatur deckt Replays ab; die Handler sind zusätzlich idempotent (Doppelzahlung → automatische Erstattung).

Tests

51 neue Tests, 1457 → 1508, alle grün.

Datei	Tests	Inhalt
src/__tests__/track-21-auth-sessions-mandantentrennung.test.ts	28	2FA-Downgrade, Rollen-Entzug, <code>staff_id</code> , Empfehlungen, Upload-Pfad
src/__tests__/track-21-sitzungswiderruf.test.ts	19	Widerrufs-Semantik, Passwortwechsel, <code>/api/auth/session-revoke</code>
src/__tests__/track-21-session-cookie-praefix.test.ts	4	Cookie-Name und -Flags in beiden Umgebungen

Alle Tests sind aus der **Angreifersicht** geschrieben: jeder ist der Versuch, mit einem gültigen Konto etwas zu erreichen, das diesem Konto nicht gehört.

Gegenprobe: mit zurückgesetztem Produktivcode (`git stash` auf die acht geänderten Produktionsdateien) fallen **29 der 51 neuen Tests** durch. Die verbleibenden 22 sind die Positivfälle — sie beschreiben Verhalten, das schon vorher richtig war, und halten fest, dass der Fix es nicht kaputt gemacht hat.

Harness-Erweiterung: `FakeSupabase` kann jetzt `auth.getUser(jwt)` (Token → User-ID über `db.authTokens`). Ohne das ließe sich der einzige Endpunkt, der sich **ohne** NextAuth-Cookie ausweist, nicht prüfen — und ein Negativtest für ein selbst erfundenes Token wäre nicht schreibbar.

Angepasster Bestandstest: `rollen-eskalation.test.ts` prüfte den Kontostand-Cache über eine **Selbst**-Herabstufung. Die ist seit CM21-04 verboten; die Assertion (Cache wird verworfen) ist unverändert geblieben und läuft jetzt über eine Herabstufung durch einen `super_admin` . Es wurde keine Abdeckung entfernt — dass Selbst- und Admin-Herabstufung abgewiesen werden, steht als eigener Test in der neuen Datei.

Operative Folgen des Deploys

1. **Alle Sitzungen enden einmalig.** Der Cookie-Name wechselt auf `__Secure-authjs.session-token` (CM21-03) — jeder muss sich neu anmelden. Kein Datenverlust, keine Rückfrage nötig. 2. **Ein Passwortwechsel meldet ab sofort alle Geräte ab**, auch das eigene (CM21-02). 3. **2FA lässt sich nicht mehr über die Kachel zurücksetzen.** Wer ein neues Telefon hat, braucht den Support. Wer 2FA neu einrichtet, muss den Code jetzt bestätigen — vorher war die Einrichtung nach dem Klick nur behauptet. 4. **Produktempfehlungen funktionieren erstmals** (CM21-06). Das ist eine Verhaltensänderung: eine Route, die immer 404 lieferte, legt jetzt Zeilen an.

Wahrheitsstand

Aussage	Stand
IMPLEMENTIERT	ja — alle sieben Befunde im Code behoben
GETESTET	ja — 51 neue Tests, 1508 gesamt grün, Gegenprobe 29/51 rot ohne Fix
TYPECHECK	ja — <code>tsc --noEmit</code> ohne Fehler
LINT	ja — 0 Fehler, 20 Warnungen (unveränderter Bestand)
BUILD	teilweise — <code>✓ Compiled successfully</code> ; die Prerender-Phase bricht lokal an fehlendem <code>SUPABASE_SERVICE_ROLE_KEY</code> ab, unverändert gegenüber HEAD
CI-GRÜN	nicht anwendbar — das Projekt hat keine CI außer dem Vercel-Build
DEPLOYED	nach <code>./deploy.sh</code> — Vercel baut automatisch
LIVE_VERIFIZIERT	nein. Kein Agent-Zugang zur Produktionsdatenbank und keine Möglichkeit, sich live anzumelden. Insbesondere ungeprüft: ob der <code>__Secure-</code> -Cookie-Name in Produktion greift und ob <code>audit_logs</code> den Widerruf tatsächlich aufnimmt. Beides ist beim ersten Login nach dem Deploy sichtbar.